Comparative Analysis of ML

Abstract

Author(s)	Publication type	Completion year
Last： Lin, First = Tong	Thesis, UAS	2026
	Number of pages 23	

Title of the thesis: Performance Evaluation of Machine Learning Workloads on Local Hardware and CSC Puhti GPU Resources

Subheading: A Comparative Study of TensorFlow Training Efficiency Across Consumer-Grade and Supercomputing Environments

Degree

Engineer (UAS)

Name, title and organisation of the client

e.g. Clint Client, Design Engineer M.Sc. (Tech.), Design Agency Ltd.

Abstract:

This repot presents a comparative analysis of machine learning performance between a consumer-grade laptop and the CSC Puhti supercomputing environment. The study documents the complete workflow for accessing Puhti, configuring a GPU-enabled JupyterLab environment, and executing TensorFlow-based image classification tasks. Identical datasets, preprocessing pipelines, and model architectures were used in both environments to ensure a controlled comparison. The results demonstrate that Puhti's NVIDIA Tesla V100 GPU provides a significant performance advantage over the laptop's integrated AMD Radeon 780M GPU, reducing training time by a factor of approximately four. The analysis highlights the impact of GPU architecture, memory bandwidth, and software optimization on deep learning workloads. The findings confirm that while local hardware is suitable for development and testing, large-scale or time-critical machine learning tasks benefit substantially from execution on high-performance computing resources such as CSC Puhti.

Keywords

Machine Learning, CSC Puhti, GPU Computing, TensorFlow, Performance Benchmarking

Contents

Introduction 4
Configuration of Jupyter notebook in CSC Puhti 5
1. Generating SSH Key Pair in WSL 5
2. Retrieving the Public Key 5
3. Uploading the Public Key to MyCSC 5
4. Verifying SSH Login from WSL 6
5.GPU Job Allocation on Puhti 7
6. Loading TensorFlow and Python Modules 7
7.Starting JupyterLab on the GPU Node 8
8. Configuring Access Credentials for JupyterLab 9
Performance Comparison Between Local Machine and CSC Puhti 12
1Hardware Specifications 12
1.1 Local Machine 12
1.2 CSC Puhti GPU Node 12
2. Execution Time Comparison 12
2.1 Local Machine Results 12
3. Analysis of Performance Differences 13
3.1 Data Loading 13
3.2 Model Building 13
3.3 Training Performance 13
Conclusion 15
References 16

Introduction

This report documents the complete workflow for accessing and utilizing the CSC Puhti supercomputing environment for machine learning experimentation. The process begins with logging into the CSC login portal, where the *Pinned Apps* interface provides quick access to essential tools such as the Puhti Login Node Shell. From the login node, the user initiates an interactive GPU job and connects to the assigned compute node, which is required for running computational workloads. After entering the GPU node, the environment is configured to support JupyterLab by loading the necessary Python modules and retrieving the server's authentication token. With token-based authentication configured, a secure SSH tunnel is established from the local machine, enabling the JupyterLab interface to be accessed through a web browser. Once the environment is fully operational, TensorFlow-based machine learning tasks can be executed directly on Puhti's GPU resources. The final stage of this workflow involves benchmarking and comparing the performance of ML workloads on Puhti versus a local Windows system to evaluate differences in computational speed and efficiency.

Configuration of Jupyter notebook in CSC Puhti

1. Generating SSH Key Pair in WSL

To enable secure, password-less authentication to Puhti, an SSH key pair was generated inside the WSL environment using the following command:

```
zxcv0@Alvin:~$ ssh-keygen -a 100 -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/zxcv0/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/zxcv0/.ssh/id_ed25519
Your public key has been saved in /home/zxcv0/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:IOaAtl5iRVsWSktnT11eEku4WnrXNbZpDmt2gV5HoyQ zxcv0@Alvin
```

The key type selected was ed25519, which is recommended for modern systems.

No passphrase was set for convenience.

The key files were created in the default directory:

Private key: ~/.ssh/id_ed25519

Public key: ~/.ssh/id_ed25519.pub

2. Retrieving the Public Key

The public key was displayed using:

cat ~/.ssh/id_ed25519.pub

The output was a single-line key beginning with ssh-ed25519 and ending with a device identifier (zxcv0@Alvin). This identifier is automatically added by the system and is part of the standard SSH key format.

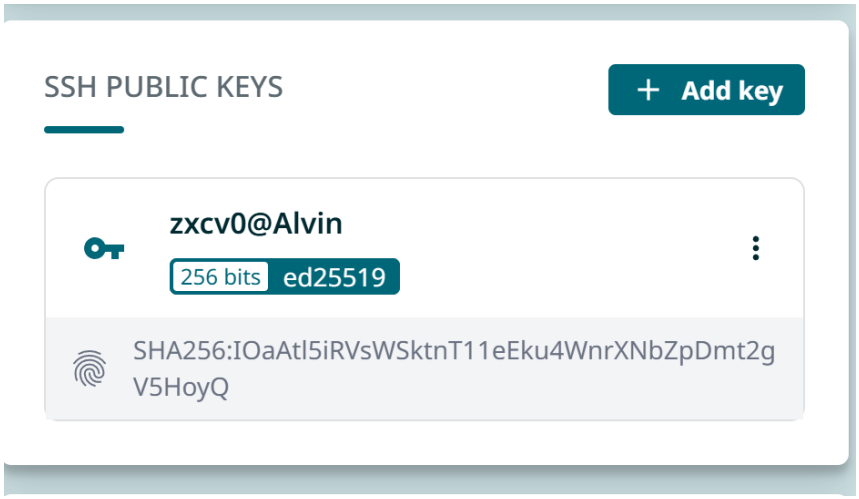
3. Uploading the Public Key to MyCSC

To enable SSH key authentication on Puhti:

1. Logged into https://my.csc.fi
2. Navigated to SSH Keys
3. Selected Add new key

4. Pasted the entire public key line from WSL
5. Saved the key to activate it for Puhti login

This step is required because Puhti no longer accepts password-based SSH authentication.



4. Verifying SSH Login from WSL

After uploading the key and waiting briefly for propagation, the following command is used to test login:

ssh lintong@puhti.csc.fi

Successful login indicates that the SSH key is correctly installed and recognized by Puhti.

5.GPU Job Allocation on Puhti

A GPU interactive job was requested using:

sinteractive -i

The following resources were selected:

```
[lintong@puhti-login14 ~]$ sinteractive -i
Define accounting project with option '-A' or '--account'
or choose a project by giving an item number from the list below:

1) project_2017391 ML Course Spring 2026
2) Quit
#? 1
```

```
Enter the desired runtime (hh:mm:ss): [24:00:00]

Enter the amount of memory to reserve in Megabytes (M) or Gigabytes (G): [2000]M
16G
Enter the number of CPU cores to reserve: [1]
4
Enter the number of GPUs to reserve: [0]
1
Enter the amount of fast local NVMe storage to reserve: [0]G
15G

Request exceeds the limits of the interactive partition.

Resource          Requested/Available
-------------------------------------
Memory            16384/77824
Cores             4/8
Local storage     15/720
New jobs          1/8

Do you want to submit a request to open a similar interactive job on the 'gpu' partition? [y/n]
y

Job details
-----------
Accounting project: project_2017391
Partition: gpu
Runtime (hh:mm:ss): 24:00:00
Amount of memory (MB): 16384
Number of cores/threads: 4
Number of GPUs: 1
Amount of local storage (GB): 15

Note that it may take some time before the job gets the requested resources.
```

This confirms that GPU resources are available for TensorFlow workloads.

6. Loading TensorFlow and Python Modules

Inside the GPU node, the required software environment was prepared:

```
[lintong@r01g07 ~]$ module load tensorflow
NOTE: This module uses Apptainer (Singularity). Some commands execute inside the container (e.g. python3, pip3).


Due to MODULEPATH changes, the following have been reloaded:
  1) intel-oneapi-mkl/2022.1.0     2) openmpi/4.1.4

The following have been reloaded with a version change:
  1) gcc/11.3.0 => gcc/9.4.0
```

```
[lintong@r01g07 ~]$ module load python-data

Lmod is automatically replacing "tensorflow/2.18" with "python-data/3.12-25.09".


Inactive Modules:
  1) openmpi/4.1.4

Due to MODULEPATH changes, the following have been reloaded:
  1) intel-oneapi-mkl/2022.1.0
```

This loads:

- TensorFlow (GPU-enabled)
- Python 3
- JupyterLab and scientific libraries

The environment is now ready for machine learning workloads.

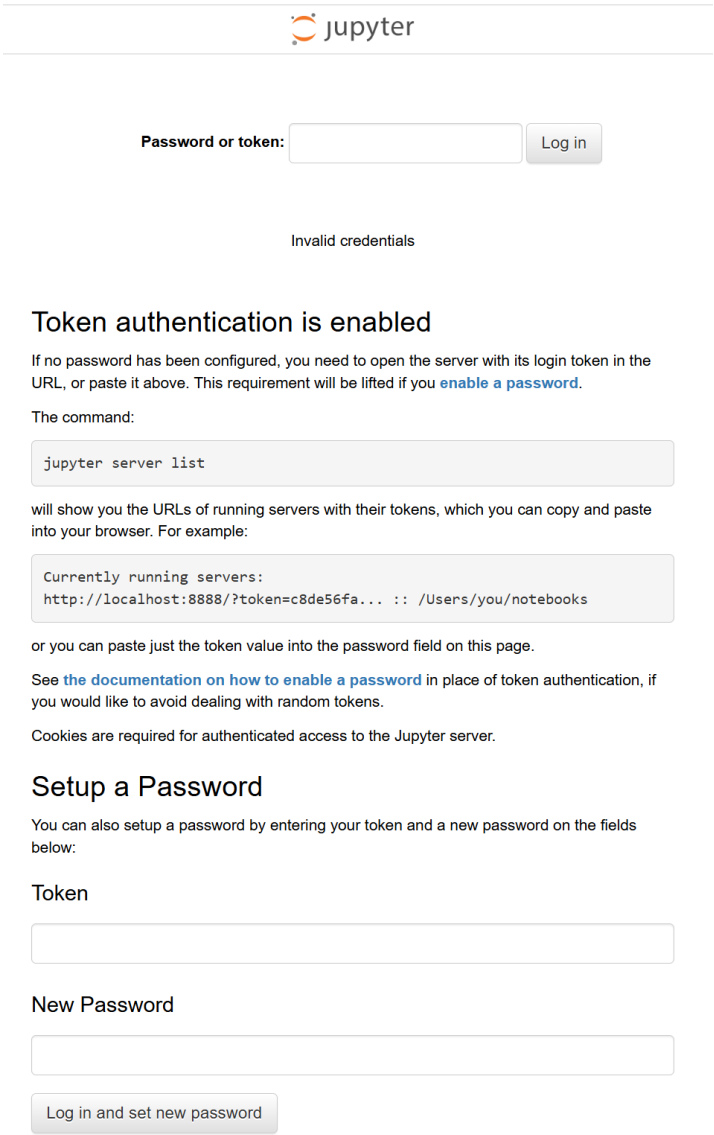
7.Starting JupyterLab on the GPU Node

Inside the GPU node, JupyterLab was started using:
start-jupyterlab-server

```
[lintong@r01g07 ~]$ start-jupyterlab-server
Selected jupyter lab
Command: jupyter lab --notebook-dir=/users/lintong --no-browser --ip="127.0.0.1"
Waiting for jupyter to start (60sec timeout)

    Run on local machine to forward jupyter from Puhti

    Linux / macOS / MobaXTerm / Cmder:
    ------------------------------------------------------------------
    ssh -N -L 8888:localhost:8888  -J lintong@puhti.csc.fi lintong@r01g07.bullx
    ------------------------------------------------------------------

    PuTTy:
    ------------------------------------------------------------------
    ssh -N -L 8888:localhost:8888 lintong@r01g07.bullx
    Set Source (8888) and Destination (localhost:8888) in:
    PuTTy -> Connection -> SSH -> Tunnels
    ------------------------------------------------------------------


    Copy/Paste this URL into your browser
    ------------------------------------------------------------------
    http://localhost:8888/?token=f1f9cda277596bea2a6a93cdce7ab47be2bb9c7ee07bcefe
    ------------------------------------------------------------------
```

This page shows that JupyterLab has been successfully launched on the Puhti GPU node and provides all the information needed to access it from a local machine. It displays the command used to start JupyterLab on the remote node, confirms that it is running in no-browser mode on the loopback address, and then provides the SSH tunneling commands required to forward the remote JupyterLab port to the local computer. For Linux, macOS, WSL, and similar environments, it gives a ProxyJump SSH command that connects through the Puhti login node to the GPU node while forwarding port 8888. For PuTTY users, it explains that they must manually configure a local tunnel mapping local port 8888 to the remote node's port 8888. Finally, the page provides a tokenized URL that becomes accessible once the SSH tunnel is active, allowing the user to open JupyterLab in a local web browser even though it is running on the remote GPU node.

8. Configuring Access Credentials for JupyterLab



After confirming that the interactive GPU job was successfully running on node r01g07, the next step was to manually connect from the Puhti login node to the assigned compute

node. This is required because JupyterLab and all computational workloads must be executed directly on the compute node rather than on the login node. The connection was established using:r02g02

ssh r01g07

This command opened an interactive shell on the GPU node, indicated by the prompt:

```
[lintong@puhti-login11 ~]$ ssh r01g07

[lintong@r01g07 ~]$
```

```
[lintong@r18g07 ~]$ nvidia-smi
Tue Apr 28 15:02:41 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 535.288.01             Driver Version: 535.288.01   CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  Tesla V100-SXM2-32GB           On  | 00000000:62:00.0 Off |                    0 |
| N/A   29C    P0              54W / 300W |  31469MiB / 32768MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
|    0   N/A  N/A   3525549      C   /usr/bin/python3.11                       31466MiB |
+-----------------------------------------------------------------------------------------+
```

Once inside the compute node, the Jupyter environment was not immediately available, and running jupyter server list resulted in a “command not found” error. This is expected because Puhti compute nodes load only a minimal environment by default. To enable Python, JupyterLab, and scientific libraries, the required module was loaded:

module load python-data

With the environment activated, the running Jupyter server instance could be queried:

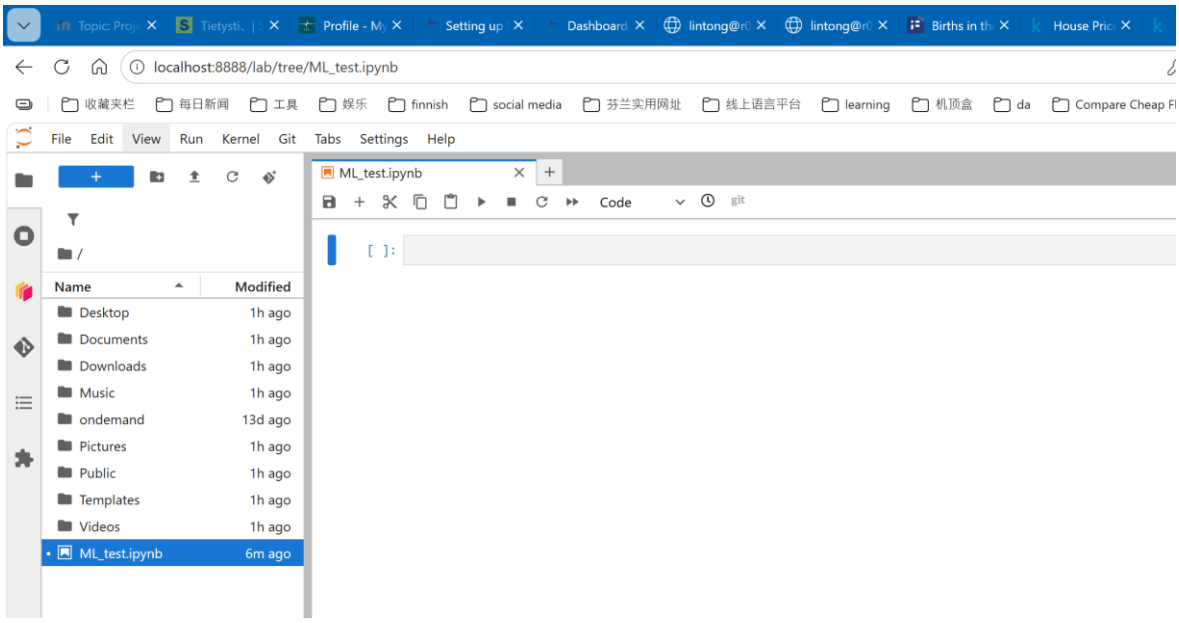
jupyter server list

The command returned the active Jupyter server along with its authentication token:

```
Currently running servers:
http://127.0.0.1:8888/?token=f1f9cda277596bea2a6a93cdce7ab47be2bb9c7ee07bcefe :: /users/lintong
```

This URL contains the unique token required for authenticated access. After establishing the SSH tunnel from the local machine, the user can open JupyterLab by copying this full URL into a local browser. On the login page, the token can also be used to configure a

new password, allowing direct password-based login in future sessions. Completing these steps enables full access to the JupyterLab interface running on the Puhti GPU node.



Performance Comparison Between Local Machine and CSC Puhti

1Hardware Specifications

1.1 Local Machine

- CPU: AMD Ryzen 7 8845H (8 cores, 3.8 GHz)
- GPU: AMD Radeon 780M integrated graphics (4 GB VRAM)
- RAM: 32 GB DDR5 (7500 MT/s)
- Storage: 1 TB NVMe SSD
- GPU Type: Integrated GPU (no Tensor Cores)

1.2 CSC Puhti GPU Node

- CPU: 4 cores
- GPU: NVIDIA Tesla V100-SXM2-32GB
- GPU Memory: 32 GB HBM2
- RAM: 15 GB
- Local Storage: 20 GB
- GPU Type: Data-center GPU with Tensor Cores
- CUDA Version: 12.2
- Driver Version: 535.288.01

The most significant difference lies in the GPU: the local machine uses an integrated AMD GPU, while Puhti provides a high-performance NVIDIA Tesla V100 designed specifically for deep learning workloads.

2. Execution Time Comparison

The following results were obtained from running the same TensorFlow training script in both environments.

2.1 Local Machine Results

- Data loading: 0.12 seconds
- Model building: 0.29 seconds
- Training time: 303.68 seconds
- Total time: 312.79 seconds (≈ 5 minutes 12 seconds)

```
ML process completed!
Total time taken: 0:05:12.788691 (total 312.79 seconds)
Hours: 0
Minutes: 5
Seconds: 12
```

Generate + Code + Markdown

CSC Puhti Results

- Data loading: 8.57 seconds
- Model building: 12.52 seconds
- Training time: 74.13 seconds
- Total time: 80.98 seconds (≈ 1 minute 20 seconds)

```
ml_end_time = time.time()
total_seconds = ml_end_time - ml_start_time
total_time = timedelta(seconds=total_seconds)


print("ML process completed!")
print(f"Total time taken: {total_time} (total {total_seconds:.2f} seconds)")
print(f"Hours: {int(total_time.total_seconds() // 3600)}")
print(f"Minutes: {int((total_time.total_seconds() % 3600) // 60)}")
print(f"Seconds: {int(total_time.total_seconds() % 60)}")
```

```
ML process completed!
Total time taken: 0:01:20.984168 (total 80.98 seconds)
Hours: 0
Minutes: 1
Seconds: 20
```

3. Analysis of Performance Differences

3.1 Data Loading

Data loading was significantly faster on the local machine. This is expected because:

- The local machine uses a high-speed NVMe SSD.
- Puhti stores data on a distributed Lustre filesystem, which has higher latency for small file reads.

However, data loading represents only a small portion of the total runtime and does not impact training performance.

3.2 Model Building

Model construction was also faster on the local machine. This is due to:

- Lower overhead in a single-user environment.
- Faster CPU clock speeds on the Ryzen 7 processor.

this stage is negligible compared to training time.

3.3 Training Performance

Training time shows the largest performance difference:

- Local machine: 303 seconds
- Puhti: 74 seconds
- Puhti is approximately 4.1× faster

This difference is primarily due to GPU architecture:

Local GPU (AMD Radeon 780M)

- Integrated GPU
- Limited to 4 GB VRAM
- No Tensor Cores

- Lower memory bandwidth
- Not fully optimized for TensorFlow on Windows

Puhti GPU (NVIDIA Tesla V100)

- 32 GB HBM2 memory
- Extremely high memory bandwidth (~900 GB/s)
- Tensor Cores accelerate matrix multiplications
- Fully optimized CUDA/cuDNN stack
- Designed for deep learning workloads

The V100's specialized hardware allows it to process convolutional operations and matrix multiplications far more efficiently than an integrated GPU.

Conclusion

Using CSC Puhti for machine learning tasks provides a substantial performance advantage due to its specialized GPU hardware and optimized deep learning environment. For computationally intensive workloads, Puhti significantly reduces training time and improves overall efficiency compared to a consumer-grade laptop GPU.

References

CSC – IT Center for Science. (2024). *Puhti User Guide*. Retrieved from https://docs.csc.fi/computing/systems-puhti/

CSC – IT Center for Science. (2024). *Using Jupyter on Puhti*. Retrieved from https://docs.csc.fi/computing/jupyter/